Programmierung für statistische Datenanalyse - Übungsblatt 1

Prof. Dr. Benjamin Buchwitz, André Münzberg

2026-10-20

Aufgabe 1 (12 P)

a) Was bedeutet *Recycling*? Beschreiben und erklären Sie diesen Begriff in eigenen Worten!

b) Aus wie vielen Elementen bestehen die Vektoren `y` und `z` jeweils?

```
y <- 1:10
z <- c(22:12, 12:22)
```

c) Gegeben sind drei Fälle, in denen `y` und `z` miteinander verrechnet werden. Begründen Sie, wann *Recycling* vorliegt und wann nicht. Zeigen Sie für alle drei Fälle, wie jeweils der letzte angegebene Wert berechnet wurde.

```
## Fall 1:
y * z
```

```
##  [1]  22  42  60  76  90 102 112 120 126 130  12  24  39  56  75  96 119 144 171
## [20] 200  21  44
```

```
## Fall 2:
y[1:5] + z[16:20]
```

```
## [1] 17 19 21 23 25
```

```
## Fall 3:
y[1:5] + z[15:20]
```

```
## [1] 16 18 20 22 24 21
```

Aufgabe 2 (6 P)

Gegeben ist:

```
1  x <- c(2, 1:2.5, 3)
2  x[2] <- 0
3  x
4  y <- c(2, 4, c(1:3), 7)
5  y[2] <- y[1] + y[2]
6  y
7  z <- c(x, y, 8)
8  z <- c(z, z)
9  length(z)
```

Was wird in

a) Codezeile 3,

b) Codezeile 6 und

c) Codezeile 9

ausgegeben und wieso? Seien Sie bei Ihrer Begründung ausführlich!

Aufgabe 3 (10 P)

Mittels `rnorm(100)` werden 100 normalverteilte Zufallszahlen erzeugt.

a) Welchen Mittelwert μ und welche Standardabweichung σ besitzen diese *Zufallszahlen*? Was müssen Sie tun, um eine Antwort auf diese Frage zu erhalten?

b) Erzeugen Sie einen Vektor `x`, der aus 100 normalverteilten Zufallszahlen mit $\mu = 1$ und $\sigma = 0,5$ besteht. Setzen Sie `set.seed(1)`.

c) Beträgt das arithmetische Mittel von `x` *wirklich* 1 und die Standardabweichung 0,5? Geben Sie die Werte gerundet auf fünf Nachkommastellen an!

d) Erzeugen Sie basierend auf den Vorgaben in Aufgabe 3b) anstelle von 100 Zufallszahlen jeweils 1.000 und 10.000 normalverteilte Zufallszahlen und benennen Sie die Vektoren `x2` und `x3`. Setzen Sie `set.seed(1)`. Berechnen Sie erneut jeweils das arithmetische Mittel und die Standardabweichung (auf fünf Nachkommastellen gerundet angeben). Was stellen Sie fest? Welche (statistische) Erklärung haben Sie hierfür?

Aufgabe 4 (10 P)

a) Schreiben Sie eine Funktion namens `standardabweichung()` mit dem Argument `x` (ein Vektor) sowie dem Rückgabewert `sigma`, die die Standardabweichung nach folgender Gleichung berechnet:

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2}.$$

b) Um was für einen Datentyp handelt es sich beim Rückgabewert der Funktion `standardabweichung()` und warum?

c) Wenden Sie Ihre Funktion `standardabweichung()` auf die Vektoren `x`, `x2` und `x3` aus Aufgabe 3 an. Geben Sie die Werte auf fünf Nachkommastellen genau an.
Erhalten Sie die gleichen Werte für σ, wenn Sie `standardabweichung()` anstelle von `sd()` für die Berechnung nutzen? Falls nein, worin besteht der Unterschied zwischen diesen beiden Funktionen und was hat er zu bedeuten? (*Hinweis:* Es ist **nicht** das Argument `na.rm` gemeint!)

Aufgabe 5 (6 P)

Gegeben ist der Vektor `eur`, der fiktive Euro-Beträge enthält:

```
1 eur <- c(123, 450, 300, 266, 321, 324, 255, 267, 220, 202, 124)
```

Die Funktion `eur_text()` gibt für Vektorelemente mit spezifischen Eigenschaften folgenden Text aus:

```
## [1] "Sie haben  450  Euro gespart. Glückwunsch!"
## [1] "Sie haben  300  Euro gespart. Glückwunsch!"
## [1] "Sie haben  266  Euro gespart. Glückwunsch!"
## [1] "Sie haben  321  Euro gespart. Glückwunsch!"
## [1] "Sie haben  324  Euro gespart. Glückwunsch!"
## [1] "Sie haben  267  Euro gespart. Glückwunsch!"
```

Wie könnte der Code der Funktion `eur_text()` aussehen? Implementieren Sie eine mögliche Funktion `eur_text()` und wenden Sie diese auf den Vektor `eur` an. **Hinweis:** `paste()`.

Aufgabe 6 (4 P)

Was bedeuten die Begriffe *Syntax* und *Semantik* und worin unterscheiden Sie sich? Erklären Sie beide Begriffe anhand des folgenden Beispiels und seien Sie ausführlich:

```r
set.seed(1)
x <- rnorm(10)
xquer <- mean(x)
v <- 0
n <- length(x)

for(idx in 1:n){
  v <- v + (x[idx] - xquer)^2
}

(1/(n - 1)) * v
```

```
## [1] 0.6093144
```

Aufgabe 7 (18 P)

Gegeben sind die folgenden drei Fälle (a)–c)) bzw. drei Code-Ausschnitte. Entscheiden Sie für jeden Fall, ob die integrierte `for`- oder `while`-Schleife und ggf. die Verzweigung notwendig ist (sind) oder ob durch Veränderung des Codes auf die entsprechende Schleife und ggf. die Verzweigung verzichtet werden kann. Begründen Sie Ihre Entscheidung.
Wenn auf die Schleife und ggf. auf die Verzweigung verzichtet werden kann, geben Sie einen entsprechenden alternativen Code an. Ändern Sie nur so viel wie nötig! *Hinweis:* Schauen Sie sich Abschnitt 4.2.1 in den Vorlesungsunterlagen https://bchwtz.github.io/bchwtz-rprg/ an.

```r
# Fall a):

set.seed(5)
v <- round(sample(x = seq(1, 100, by = 0.1), size = 25, replace = TRUE))
vres <- v
vi <- 1:length(v)

for(e in vi){
  if((v[e] %% 2) != 0){
    vres[e] <- v[e] + 1
  }else{
    vres[e] <- v[e]
  }
}
vres
```

```r
# Fall b):

x <- seq(from = -4, to = 4, by = 0.1)
sigma <- 1
mu <- 1
y <- 0

for(idx in 1:length(x)){
  factor1 <- (1/sqrt(2 * pi * sigma^2))
  factor2 <- exp(-(x[idx] - mu)^2 /(2 * sigma^2))
  res <- factor1 * factor2
  y <- c(y, res)
}
y <- y[-1]
y
```

```r
# Fall c):

x <- seq(from = 4, to = 0, by = -0.1)
sigma <- 1
mu <- 1
y <- 0
idx <- 1

while(x[idx] >= 2){
  factor1 <- (1/sqrt(2 * pi * sigma^2))
  factor2 <- exp(-(x[idx] - mu)^2 /(2 * sigma^2))
  res <- factor1 * factor2
  y <- c(y, res)
  idx <- idx + 1
}
y <- y[-1]
y
```